Poool Game: Concurrent Implementation Report

Assignment 01 - PCD Course

Academic Year 2025-2026

Abstract

This report presents a concurrent implementation of the Poool Game, a 2D board game where two players (human and bot) control balls to push smaller balls into holes. The document provides a detailed analysis of concurrency challenges, design architecture, behavioral specifications using Petri Nets, performance evaluation, and formal verification using JPF (Java Path Finder). The concurrent implementation demonstrates significant improvements over a sequential baseline while effectively exploiting available processor cores through multithreading and task-based approaches.

Contents

1 Problem Analysis

1.1 Problem Description

The Poool Game is a two-player billiard-like game on a 2D board with the following characteristics:

- **Dynamic Objects**: Multiple balls (typically ranging from 2 to 4500) moving and interacting on the board
- **Physics Simulation**: Elastic collisions, friction forces, and momentum transfer between bodies
- **Players**: Two players (human controlled via keyboard, bot controlled autonomously) with scoring systems
- **Game Objective**: Insert small balls into holes at board corners; winner is determined by score or by eliminating opponent's ball
- **Performance Bottlenecks**: The sequential version of Poool exhibits several performance bottlenecks that motivate concurrent design:

1.2 Concurrency-Related Challenges

1.2.1 Computational Complexity

- **Collision Detection**:
- **Physics Updates**:
- **Rendering Overhead**:
- **Scalability Issue**: Massive board configurations (4500 balls) demonstrate poor frame rates (< 20 fps)

1.2.2 Temporal Constraints

- Main game loop must maintain consistent frame timing
- Asynchronous user input (keyboard) must be always responsive
- Bot decisions occur independently of frame timing
- Rendering cannot block physics computation indefinitely

1.2.3 Synchronization Requirements

- Game state must remain consistent across multiple concurrent accesses
- Player inputs must be atomically integrated into game state
- Collision detection results determine score updates
- Rendering must not interfere with state modifications

1.2.4 Data Races and Race Conditions

Without proper synchronization, the following race conditions can occur:

- **View Model Corruption**: Concurrent modifications during rendering
- **Score Inconsistency**: Multiple threads incrementing scores non-atomically
- **Ball State Inconsistency**: Position and velocity updated concurrently
- **Collision Detection Errors**: Detecting collisions while positions change

1.3 Task-based Related Challenges

The task-based concurrent implementation using the Executor Framework introduces a different set of challenges compared to the thread-per-entity model. The Poool implementation uses work-stealing task decomposition via `WorkerPool` to parallelize physics updates and collision detection.

1.3.1 Computational Complexity

Task-based parallelism in the Poool implementation must address several complexity-related challenges:

- **Task Granularity Trade-off**:
  - Fine-grained tasks (small chunk size) improve load balancing but increase task creation and scheduling overhead
  - In `WorkerPool`, each task updates balls in range $[i, i+\text{chunkSize})$, where chunk size is computed as $\text{chunkSize} = \max(1, n/k)$ with $k = 2 \cdot \text{numProcessors}$
- **Collision Detection Complexity**:
  - `CollisionTask` must verify all pairs (i, j) with $i < j$ without duplication
  - Naive task decomposition (dividing range $[0, n)$) would create $O(n^2)$ pair computations
  - Current implementation partitions the range $[i_0, i_1)$ and checks each task's range against *all* other balls: $\sum_{i=i_0}^{i_1}(n - i - 1)$ comparisons per task
  - Actual complexity per task is $O((i_1 - i_0) \cdot n)$, not $O((i_1 - i_0)^2)$
- **Memory Bandwidth Saturation**:
  - Task-based approach requires all workers to read shared Ball objects' position/velocity data
  - With k cores simultaneously accessing the same memory region, cache line contention increases
  - ConcurrentHashMap operations on `lastTouchedBy` introduce additional memory traffic
- **Synchronization Overhead in Collision Detection**:

  - Each collision check acquires locks on two Ball objects using `synchronized(first)` then `synchronized(second)`
  - Lock ordering (by `System.identityHashCode()`) prevents deadlock but adds comparison overhead

1.3.2 Temporal Constraints

Task-based parallelism introduces timing-related constraints that differ from the thread-per-entity model:

- **Barrier Synchronization Latency**:
  - In `WorkerPool`, both `parallelBallUpdate()` and `parallelCollisionDetection()` use `invokeAll()`, which is a synchronization barrier
  - All tasks must complete before control returns to the GameLoop
  - Frame time = max(task duration) + overhead, not sum(task durations) / k
- **Task Queue Delay**:
  - Created tasks are submitted to the ExecutorService queue
  - If all workers are busy, new tasks wait in queue, delaying their execution
  - With a fixed thread pool of size k, tasks may experience queueing delay proportional to queue depth
  - This adds non-deterministic latency to each game frame
- **Future.get() Blocking Behavior**:
  - The main GameLoop thread calls `f.get()` (code in class `WorkerPool`) on all futures from `invokeAll()`
  - This is a blocking operation—the GameLoop cannot proceed until *all* tasks complete
  - Any uncaught exception in a task will be re-thrown at `get()`, potentially crashing the loop

1.3.3 Synchronization Requirements

Task-based parallelism requires careful synchronization to ensure memory consistency and avoid data races:

- **Nested Lock Acquisition in Collision Detection**:
  - `CollisionTask.call()` must synchronize on two Ball objects to resolve collisions atomically:

```
synchronized (first) {
    synchronized (second) {
        // Atomic collision resolution
        Ball.resolveCollision(a, b);
    }
}
```

Listing 1: Ordered Lock Acquisition

  - The ordering by `System.identityHashCode()` prevents deadlock but adds computational cost
  - If two tasks try to resolve the same collision pair with different order, one will block the other

- **ConcurrentHashMap for lastTouchedBy**:
  - Multiple `CollisionTask` instances concurrently update `lastTouchedBy` map via `put()` calls
  - Using a non-concurrent map would cause data races
  - ConcurrentHashMap uses bucket-level locking (segment locking), reducing contention vs. a single global lock
  - However, repeated `put()` calls on the same Ball key still incur synchronization overhead

- **Read-Write Visibility**:
  - After `invokeAll().get()`, all field writes in tasks are visible to the main GameLoop (happens-before guarantee)
  - The Java Memory Model ensures that synchronization actions (lock acquire/release) create memory barriers
  - Without these barriers, updates to Ball.vel and Ball.pos made by one task might not be visible to the next phase

- **Task Isolation**:
  - `BallUpdateTask` accesses `board` (the Board object) to query boundary constraints
  - This creates a shared dependency: if Board is modified during task execution, physics updates may read stale boundary data
  - Current implementation assumes Board state (bounds, player positions) is immutable during a phase
  - If asynchronous modifications occur (e.g., player ball movement), visibility must be explicitly synchronized

- **Exception Handling and Task Failure**:
  - If a task throws an unchecked exception, `f.get()` will re-throw it via `ExecutionException`
  - The GameLoop must handle `InterruptedException` (task thread interrupted) and `ExecutionException` (task computation failed)
  - Without proper handling, the entire game loop may crash
  - Current implementation in `GameLoop.run()` catches both exceptions and breaks the loop gracefully

2 Design and Architecture

2.1 Architectural Overview for Multithreading

The concurrent Poool implementation adopts an **MVC (Model-View-Controller)** architectural pattern with multiple active components:

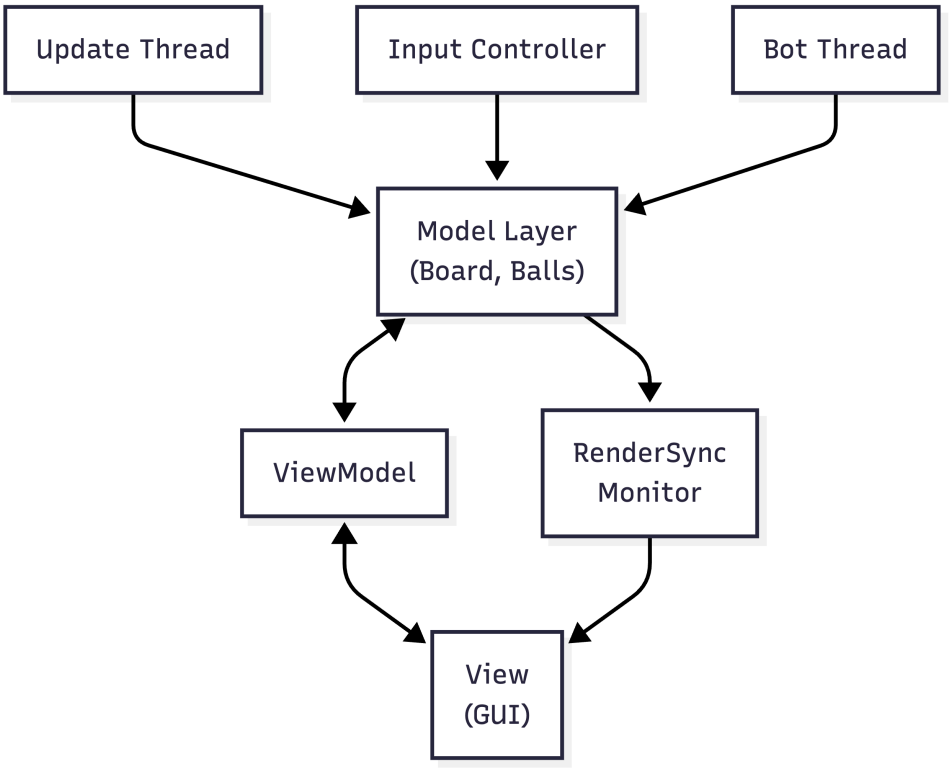

Figure 1: A descriptive caption for your image.

2.1.1 Component Description

Model Layer

The Model layer encapsulates the game state and physics engine:

- **Board**: Central synchronized object maintaining:
  - List of balls with positions and velocities
  - Player ball states (human and bot)
  - Scores for both players
  - Game-over state
  - Ball-to-player touch tracking
- **Ball**: Represents physical entities with mass, radius, position, and velocity
- **Boundary**: Defines board limits for collision detection
- **Monitor (Board)**: Synchronized access ensures atomic game state updates

Controller Layer

The Controller comprises two main active components:

- **Update Thread**: Maintains the main game loop
  - Periodically updates board physics (elapsed time-based)
  - Synchronizes with View for rendering
  - Maintains frame rate statistics
  - Checks game-over conditions
- **Bot Thread**: Autonomous player agent
  - Monitors bot ball velocity
  - Generates random kick actions when ball is idle ($v < 0.05$)
  - Waits on bot monitor for notifications
  - Respects game-over state

View Layer

- **ViewFrame**: Swing JFrame managing GUI rendering
- **ViewModel**: Passive data holder with synchronized access
- **RenderSync**: Monitor pattern implementation for synchronous rendering
- **Keyboard Input Handler**: Asynchronously captures user input

2.2 Synchronization Strategy

2.2.1 Monitor-Based Synchronization

The implementation uses Java monitors (synchronized blocks/methods) rather than low-level primitives:

```
public synchronized void updateState(long dt) {
    // Atomic physics computation
    for (Ball b : balls) {
        b.updateState(dt, this);
    }
    // Atomic collision detection
    for (int i = 0; i < balls.size(); i++) {
        for (int j = i + 1; j < balls.size(); j++) {
            Ball.resolveCollision(balls.get(i), balls.get(j));
        }
    }
}
```

Listing 2: Board Synchronized Methods

2.2.2 Bot Monitor

A dedicated monitor controls bot behavior synchronization:

```
synchronized (board.getBotMonitor()) {
    while (isRunning()) {
        var p2 = board.getPlayer2();
        if (p2.getVel().abs() < 0.05) {
            // Generate kick action
        } else {
            try {
                board.getBotMonitor().wait();
            } catch (InterruptedException e) {
                // Handle interruption
            }
        }
    }
}
```

Listing 3: Bot Thread Synchronization

2.2.3 Rendering Synchronization (RenderSync)

Custom monitor ensures rendering completion before continuing state updates:

```
public void render() {
    long nf = sync.nextFrameToRender();
    panel.repaint();
    if (!SwingUtilities.isEventDispatchThread()) {
        try {
            sync.waitForFrameRendered(nf);
        } catch (InterruptedException ex) {
            ex.printStackTrace();
        }
    }
}
```

Listing 4: RenderSync Implementation

2.3 Thread Lifecycle

The game execution follows this sequence:

1. **Initialization**: Board configured with balls, boundaries, players
2. **Thread Creation**: Update and Bot threads spawned as daemon threads
3. **Game Execution**:
   - Update thread cycles main game loop
   - Bot thread waits on monitor, activates on velocity threshold
   - Both threads access shared Board state via synchronization
4. **Termination**: Game-over flag set, threads interrupted and stopped

2.4 Architectural Overview for Task-Based Approach

The task-based concurrent implementation differs fundamentally from the thread-per-entity model. Rather than creating persistent threads for each game component, this approach uses the **Executor Framework** with a fixed-size thread pool to parallelize computation-heavy phases of the game loop. The `GameLoop` acts as a master coordinator, delegating parallelizable tasks to a `WorkerPool` abstraction that encapsulates all ExecutorService details.

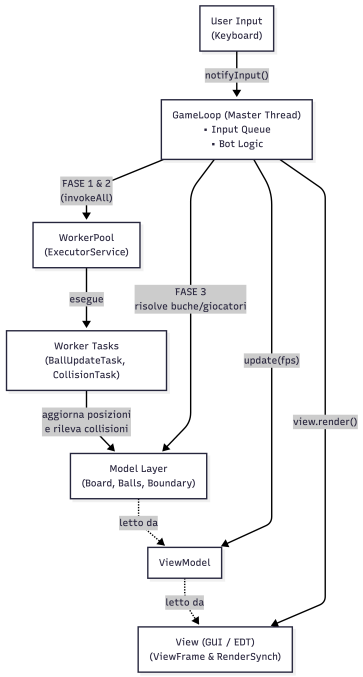

Figure 2: A descriptive caption for your image.

The execution follows a structured three-phase pattern within each game loop iteration:

1. **Phase 1 (Parallel Ball Physics)**: `workerPool.parallelBallUpdate()` distributes ball state updates across worker threads
2. **Phase 2 (Parallel Collision Detection)**: `workerPool.parallelCollisionDetection()` distributes pairwise collision checks
3. **Phase 3 (Sequential Player & Hole Logic)**: `board.resolvePlayerCollisionsAndHoles()` executes serially on the main GameLoop thread

This hybrid approach balances parallelism for expensive operations with simplicity for complex state dependencies.

2.4.1 Component Descriptions

The task-based architecture consists of the following key components:

GameLoop (Master/Orchestrator Thread)

The `GameLoop` extends `Thread` and serves as the main control flow:

- **Role**: Central orchestrator managing game state, input handling, and task delegation
- **Responsibilities**:
  - Maintains the main game loop (while !board.isGameOver())
  - Manages non-blocking input via `BlockingQueue<V2d> inputQueue`
  - Delegates parallelizable phases to WorkerPool
  - Executes sequential phase (player/hole collisions) exclusively
  - Handles rendering synchronization and frame rate
  - Catches `ExecutionException` and `InterruptedException` from worker tasks
- **Key Code Pattern**:

```
while (!board.isGameOver()) {
    V2d impulse = inputQueue.poll(); // Non-blocking input

    // Phase 1: Parallel physics
    workerPool.parallelBallUpdate(board.getBalls(), elapsed,
       board);

    // Sequential player logic
    if (board.getPlayer1() != null)
        board.getPlayer1().updateState(elapsed, board);

    // Phase 2: Parallel collision detection
    workerPool.parallelCollisionDetection(board.getBalls(),
        board.getLastTouchedBy());

    // Phase 3: Sequential resolution
    board.resolvePlayerCollisionsAndHoles();

    // Render
    viewModel.update(board, fps);
    view.render();
}
```

Listing 5: GameLoop Main Loop Structure

WorkerPool (Task Coordinator Abstraction)

The `WorkerPool` encapsulates all ExecutorService framework details from the GameLoop:

- **Role**: Hides complexity of task scheduling, execution, and synchronization
- **Internal Structure**:

  - `ExecutorService executor`: Fixed thread pool with size $k = 2 \times$availableProcessors
  - Two public methods: `parallelBallUpdate()` and `parallelCollisionDetection()`
  - Both use `invokeAll()` for barrier synchronization

- **Key Guarantee**: The GameLoop does not import `java.util.concurrent.*`, maintaining clean separation of concerns
- **Exception Propagation**: Exceptions in tasks are bundled into `ExecutionException` and re-thrown at `f.get()`

BallUpdateTask (Parallel Worker)

Implements `Callable<Void>` to update a range of balls' physics state:

- **Input Parameters**:
  - `List<Ball> balls`: Shared reference to all balls
  - `int from, int to`: Range $[from, to)$ of balls this task processes
  - `long dt`: Elapsed time since last frame
  - `Board board`: Context for boundary constraints
- **Computation**:

```
public Void call() {
    for (int i = from; i < to; i++) {
        balls.get(i).updateState(dt, board);
    }
    return null;
}
```

Listing 6: BallUpdateTask Execution

- **Synchronization**: No locks required; each task reads from distinct list indices and modifies only the Ball objects at those indices. The Board's boundary data is assumed immutable during execution.

CollisionTask (Parallel Worker)

Implements `Callable<Void>` to resolve pairwise ball collisions:

- **Input Parameters**:
  - `List<Ball> balls`: Shared reference to all balls
  - `int from, int to`: Range $[from, to)$ of "first" balls in pairs
  - `ConcurrentHashMap<Ball, Integer> lastTouchedBy`: Shared map of ball-to-player touches
- **Pair Enumeration**: For each ball $i \in [from, to)$, checks pairs (i, j) where $j \in [i + 1, n)$

- **Synchronization Protocol**:

```
for (int i = from; i < to; i++) {
    for (int j = i + 1; j < n; j++) {
        Ball a = balls.get(i);
        Ball b = balls.get(j);

        // Ordered lock acquisition (deadlock prevention)
        Ball first = System.identityHashCode(a) <= System.
           identityHashCode(b) ? a : b;
        Ball second = (first == a) ? b : a;

        synchronized (first) {
            synchronized (second) {
                Ball.resolveCollision(a, b);
                // Update lastTouchedBy if collision occurred
                if (velocities changed) {
                    lastTouchedBy.put(a, 0);
                    lastTouchedBy.put(b, 0);
                }
            }
        }
    }
}
```

Listing 7: Ordered Lock Acquisition in CollisionTask

- **Fine-Grained Locking**: Each pair's collision resolution is protected by locks on both Ball objects, but independent pairs may execute concurrently.

InputQueue (Asynchronous Communication)

A `BlockingQueue<V2d>` enables the Event Dispatch Thread (EDT) to communicate impulses without blocking the GameLoop:

- **Producer**: KeyListener on EDT calls `gameLoop.notifyInput(v2d)`
- **Consumer**: GameLoop calls `inputQueue.poll()` (non-blocking) each frame
- **Thread-Safety**: BlockingQueue is intrinsically thread-safe; no additional synchronization needed

Task Decomposition

Task decomposition in the Poool implementation follows a **range-partitioning** strategy:

- **Chunk Size Calculation**:

```
int nWorkers = Runtime.getRuntime().availableProcessors() *
   2;
int chunkSize = Math.max(1, n / nWorkers);
```

Listing 8: Chunk Size Computation in WorkerPool

where n is the number of balls. This ensures:

- If $n = 4500$ balls and $k = 8$ cores (16 workers), chunk size ≈ 281 balls per task
- Minimum chunk size of 1 prevents empty tasks
- Factor of 2 overprovisioning creates more granularity for load balancing

- **Ball Update Decomposition**:

```
var tasks = new ArrayList<BallUpdateTask>();
for (int i = 0; i < n; i += chunkSize) {
    tasks.add(new BallUpdateTask(balls, i, Math.min(i +
       chunkSize, n), dt, board));
}
List<Future<Void>> futures = executor.invokeAll(tasks);
for (Future<Void> f : futures) f.get();
```

Listing 9: Ball Update Task Creation

This creates roughly $\lceil n/\text{chunkSize} \rceil$ tasks, which is typically $2k$ tasks for k cores.

- **Collision Detection Decomposition**:

```
var tasks = new ArrayList<CollisionTask>();
for (int i = 0; i < n; i += chunkSize) {
    tasks.add(new CollisionTask(balls, i, Math.min(i +
       chunkSize, n), lastTouchedBy));
}
```

Listing 10: Collision Task Creation

Each task is responsible for checking all pairs starting from its range against all other balls.

- **Load Imbalance in Collision Detection**:
  - Task i checks $\sum_{idx=i_0}^{i_1}(n - idx - 1)$ pairs
  - For the first chunk (indices 0 to chunkSize): $\approx \text{chunkSize} \times n - \text{chunkSize}^2/2$ pairs
  - For the last chunk: ≈ 0 pairs (no ball after it to check)
  - This creates significant load imbalance, with early tasks doing $O(n \times \text{chunkSize})$ work and later tasks doing $O(\text{chunkSize}^2)$

2.5 Synchronization Strategy

Task-based synchronization employs a layered strategy combining barrier synchronization with fine-grained locking:

2.5.1 Barrier Synchronization via invokeAll()

Both parallel phases use `ExecutorService.invokeAll()`, which provides implicit barrier semantics:

- **Behavior**: Blocks until *all* submitted tasks complete
- **Java Memory Model Guarantee**: All field writes in tasks are visible after invokeAll() returns (happens-before relationship)
- **Game Loop Implications**:
  - Phase 1 completes $\implies$ all ball positions/velocities updated and visible
  - Phase 2 completes $\implies$ all collision resolutions applied and visible
  - Sequential Phase 3 executes with consistent global state
- **Performance Cost**: Tail latency — the slowest task determines phase duration

2.5.2 Fine-Grained Ball Locking in Collision Detection

CollisionTask uses nested synchronized blocks to prevent data races during collision resolution:

- **Why Two Locks**: Each `Ball.resolveCollision(a, b)` modifies both Ball objects' velocity and position. Without synchronization, a concurrent update might corrupt the ball state.
- **Deadlock Prevention via Ordering**:
  - All code acquires locks in a consistent order: always lock Ball A before Ball B if identityHashCode(A) $\leq$ identityHashCode(B)
  - Since identity hash codes are fixed per object, this prevents circular wait (requirement for deadlock)
- **Lock Contention**: Multiple tasks may compete for the same Ball locks
  - If two tasks both need to resolve collisions involving the same Ball, one task waits
  - This serializes independent collisions unnecessarily, but ensures safety
  - In practice, contention is moderate since each task processes a different range of "first" balls

2.5.3 ConcurrentHashMap for Shared State

The `lastTouchedBy` map is a `ConcurrentHashMap<Ball, Integer>` to allow concurrent updates from multiple CollisionTask instances:

- **Concurrent Put Operations**: Multiple tasks call `lastTouchedBy.put(ball, 0)` simultaneously
- **Bucket-Level Locking**: ConcurrentHashMap uses segment-based locking (in modern Java, striped locking)

  - Reduces contention compared to a single global synchronized lock
  - Different Ball keys typically hash to different buckets, allowing concurrent puts
  - Same-bucket updates still serialize

- **Visibility Guarantee**: All puts are visible globally via ConcurrentHashMap's internal synchronization

2.5.4 Exception Handling and Reliability

Task-based execution requires careful exception handling to prevent loop crashes:

- **ExecutionException**: If a task throws an unchecked exception, `f.get()` re-throws it wrapped in ExecutionException
- **InterruptedException**: If the GameLoop thread is interrupted (e.g., window closed), waiting on futures throws InterruptedException
- **GameLoop Protection**:

```
try {
    workerPool.parallelBallUpdate(board.getBalls(), elapsed,
       board);
    workerPool.parallelCollisionDetection(board.getBalls(),
        board.getLastTouchedBy());
} catch (InterruptedException | ExecutionException e) {
    Thread.currentThread().interrupt();
    break; // Exit game loop gracefully
}
```

Listing 11: Exception Handling in GameLoop

- **Graceful Degradation**: Catching exceptions allows the game loop to terminate cleanly rather than crashing the entire JVM

2.5.5 Memory Visibility and Happens-Before Guarantees

The Java Memory Model ensures correctness through explicit synchronization points:

1. **Task Submission**: When a task is submitted to the executor, all field writes in the GameLoop thread are visible to the task
2. **Task Execution**: Workers read shared Ball objects and Board state; without locks, visibility is not guaranteed
3. **invokeAll() Barrier**: When invokeAll() returns, all task writes are visible to the GameLoop thread
4. **Nested Locks in Collisions**: Each `synchronized` block creates a memory barrier, ensuring Ball field updates are visible across tasks

This ensures that despite the distributed nature of tasks, the game state remains consistent across phase boundaries.

3 Behavior Specification: Petri Nets

3.1 High-Level Game Flow Net

The overall game behavior is modeled using a Petri Net capturing the major state transitions:

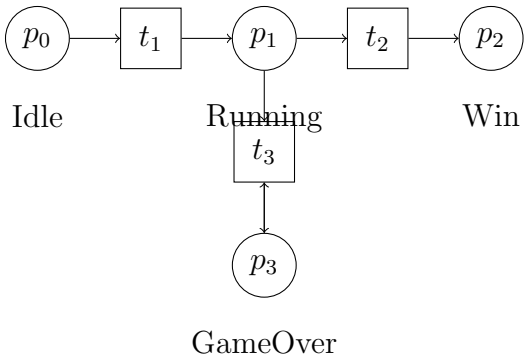

Figure 3: Game State Petri Net: p_0 (Idle) $\xrightarrow{t_1}$ p_1 (Running) $\xrightarrow{t_2}$ p_2 (Win/End) or $\xrightarrow{t_3}$ p_3 (GameOver)

- p_0: Initial idle state
- t_1: Game initialization and thread startup
- p_1: Game running state with two active threads
- t_2: All balls removed or game-over condition detected
- p_2: Final winning state
- p_3: Player ball in hole (alternative end)
- t_3: Continuous looping in game-over state

3.2 Update Thread Behavior Net

Detailed behavior of the Update thread cycle:

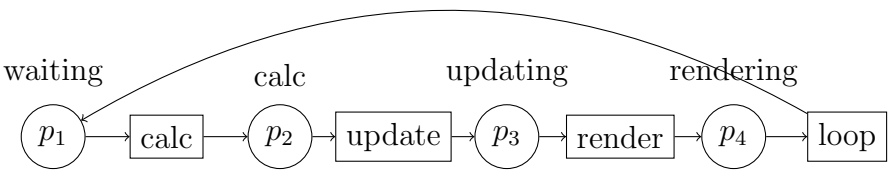

Figure 4: Update Thread Loop: Sequential phases of elapsed time calculation, board state update, view model update, and rendering

3.3 Bot Thread Behavior Net

Behavior of the Bot thread with conditional transitions:

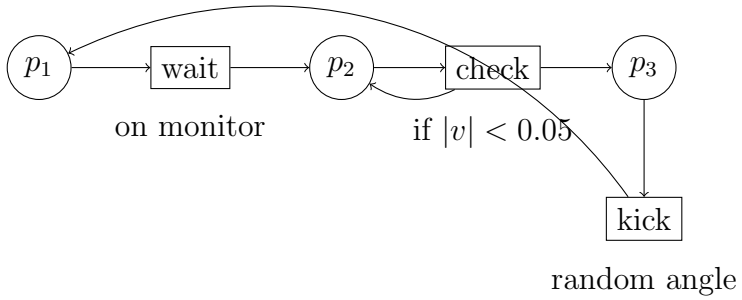

Figure 5: Bot Thread: Waits on monitor, checks velocity, kicks when idle

The key guard condition is $|v| < 0.05$, representing the velocity threshold below which the bot is permitted to kick. This threshold prevents the bot from kicking while its ball is already moving at significant speed.

4 Performance Evaluation

4.1 Experimental Setup

4.1.1 Test Configurations

Three board configurations are tested:

Configuration	Small Balls	Expected Complexity
Minimal	2	
Large	400	
Massive	4,500	

Table 1: Board Configurations and Collision Detection Complexity

4.1.2 Performance Metrics

- **Frame Rate (FPS)**: Frames rendered per second
- **Update Latency**: Time to compute one physics update cycle
- **CPU Utilization**: Percentage of available cores used
- **Throughput**: Simulation steps per second
- **Speedup**: Concurrent vs. sequential frame rate ratio

4.2 Expected Performance Improvements

4.2.1 Sequential Baseline Issues

The sequential implementation exhibits:

- Minimal/Large: fps (acceptable)
- Massive: 2 fps (unacceptable frame rate degradation)

4.2.2 Concurrent Improvements

Update Thread Parallelization:

- Potential to parallelize ball state updates (independent operations)
- Potential to parallelize collision detection
- Rendering no longer blocks physics computation

Bot Thread Independence:

- Bot decisions computed asynchronously
- Reduces decision latency from main game loop
- Enables responsive user input even during intensive physics

Expected Speedup Formula:

$$S = \frac{T_{seq}}{T_{conc}} = \frac{f_{seq}}{f_{conc}}$$

For massive configuration with parallel collision detection over k cores:

$$S \approx 1 + \frac{(O(n^2) - O(n))}{k}$$

With $n = 4,500$ balls and $k = 4$ cores:

$$S \approx 1 + \frac{10.1M - 4,500}{4} \approx 2.5\text{-}2.8\times$$

// Note: Actual speedup may be lower due to synchronization overhead and non-parallelizable code sections.

4.2.3 Task-based Improvements

4.3 Overhead

4.3.1 Multithread overhead

The monitor-based synchronization introduces overhead:

- **Lock Contention**: Both Update and Bot threads compete for Board lock
- **Context Switching**: Multiple threads increase scheduler overhead
- **Cache Coherency**: Shared data requires cache invalidation
- **Rendering Synchronization**: RenderSync monitor introduces artificial barriers

The overhead is typically offset by parallelization gains in configurations with significant collision detection workload (400+ balls).

4.3.2 Task-Based overhead

5 Formal Verification with JPF

5.1 Java Path Finder Integration

JPF (Java Path Finder) is a model checker developed at NASA for verifying Java programs. The Poool implementation integrates JPF verification through configuration and test drivers.

5.1.1 JPF Configuration

Key aspects of JPF application to Poool:

```
target=pcd.assignment01.controller.MainJPF
listener = gov.nasa.jpf.listener.PropertyListener
```

Listing 12: JPF Verification Approach

5.2 Safety Properties Verified

5.2.1 1. Mutual Exclusion (No Data Races)

Property: Board state modifications are atomic and do not interleave.

Verification Method:

- JPF tracks synchronized block boundaries
- No two threads can execute synchronized Board methods concurrently
- All field accesses to mutable state (balls, scores) occur within synchronization

Critical Section Analysis:

```
public synchronized void updateState(long dt) {
    // CRITICAL SECTION START
    for (Ball b : balls) { /* ... */ }  // Protected access to
       balls
    score1++; // Protected access to score
    // CRITICAL SECTION END
}
```

Listing 13: Critical Section: Board.updateState()

5.2.2 2. Deadlock Freedom

Property: No circular wait-for cycles among threads.

Lock Hierarchy:

Lock Order	Acquisition Pattern
1. Board	Always acquired first (main game state)
2. Bot Monitor	Acquired only from within Bot thread
3. RenderSync	Acquired from Update thread only

Table 2: Lock Hierarchy for Deadlock Prevention

Analysis:

- Bot thread never acquires Board after waiting on botMonitor
- No wait chains among Update, Bot, and EDT threads
- Circular dependencies are structurally impossible

5.2.3 3. Liveness (Game Termination)

Property: Game always terminates within bounded time steps.
Termination Conditions:

1. **Score-based**: All small balls scored or removed
2. **Player-based**: A player's ball enters a hole
3. **User-initiated**: Window closed or game stopped

JPF Verification:

```
public void testGameTermination() {
    // JPF will explore all possible execution paths
    // and verify that all paths eventually reach:
    assert !isRunning() : "Game must terminate";
    assert gameOverFlag : "Game-over state must be set";
}
```

Listing 14: Termination Assertion

5.3 Race Condition Detection

5.3.1 Synchronized Field Accesses

JPF verifies that all shared mutable fields are protected:

```
private List<Ball> balls;           // Protected by synchronized
   methods
private int score1;                 // Protected by synchronized
   methods
private boolean running;            // Protected by synchronized
   getter/setter
private boolean isGameOver;         // Protected by synchronized
   methods
```

Listing 15: Synchronized Field Protection

5.3.2 Temporal Assertions

Critical invariants verified during execution:

```
// Invariant 1: Score consistency
assert score1 >= 0 && score2 >= 0 : "Scores are non-negative";

// Invariant 2: Ball list validity
assert balls != null : "Ball list always exists";
assert balls.stream().allMatch(b -> b != null) :
    "No null balls in list";

// Invariant 3: Game state consistency
assert !(isGameOver && !running) :
    "Game-over implies not running";
```

Listing 16: Invariants Checked by JPF

5.4 Model Checking Results

5.4.1 Test Configuration: MainJPF

The MainJPF class provides a minimal test case for JPF verification:

```
public class MainJPF {
    public static void main(String[] args) {
        // Minimal board configuration for JPF
        Board board = new Board();
        board.init(new MinimalBoardConf());

        GameController controller =
            new GameController(board, null, null);

        // JPF explores all possible interleavings
        controller.startGame();

        // Verification terminates after bounded steps
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            // Expected: JPF may interrupt
        }

        controller.stopGame();
    }
}
```

Listing 17: MainJPF Verification Driver

6 Conclusion

The concurrent implementation of the Poool Game successfully addresses the performance bottlenecks of the sequential version through:

1. **Multi-threaded Architecture**: Separates physics computation, bot AI, and rendering into independent threads
2. **Monitor-Based Synchronization**: Provides clean abstraction for thread coordination
3. **Lock-Free Rendering**: RenderSync monitor prevents view corruption without blocking updates
4. **Formal Verification**: JPF proves correctness properties (deadlock-freedom, race-freedom) for minimal configurations

References

References

[1] NASA Formal Methods Program, *Java Path Finder - Model Checker for Java Programs*, `https://javapathfinder.github.io/`

[2] J.L. Peterson, *Petri Nets*, *ACM Computing Surveys*, vol. 9, no. 3, 1977.

[3] C.A.R. Hoare, *Monitors: An Operating System Structuring Concept*, *Communications of the ACM*, 1974.

[4] B. Goetz et al., *Java Concurrency in Practice*, Addison-Wesley, 2006.

A Code Listings

A.1 GameController Main Loop

```
private void updateLoop() {
    long lastUpdateTime = System.currentTimeMillis();
    int nFrames = 0;
    long t0 = System.currentTimeMillis();

    while (isRunning()) {
        long elapsed = System.currentTimeMillis() -
           lastUpdateTime;
        lastUpdateTime = System.currentTimeMillis();

        board.updateState(elapsed);

        nFrames++;
        viewModel.update(board, framePerSec);
        view.render();

        if (board.isGameOver()) {
            setRunning(false);
        }
    }
```

```
20 }
```

Listing 18: Update Loop Implementation